

CODVO.AI

TAKE-HOME ASSIGNMENT · DATA SCIENCE ARCHITECT

Predictive Equipment Failure — Industrial Pump Fleet

Build an end-to-end machine-learning solution that forecasts when industrial pumps are heading toward failure, using multivariate time-series sensor data.

ROLE

Data Science Architect

EFFORT

~3 days

PRESENTATION

1 hour (45 + 15 min)

DUE

Presentation day

1 Scenario

A process plant runs a fleet of motor-driven centrifugal pumps. Unplanned pump failures stop production lines, and reactive repairs are expensive and slow.

The reliability team has exported several years of historian data: a run-to-failure record for each pump in the fleet, sampled roughly hourly, across ~20 sensor and operating channels. They want to move from **reactive** repair to **proactive** maintenance — knowing, with enough lead time, that a specific pump is degrading toward failure so a work order can be raised before it breaks.

You have been asked, as the data science architect, to design and build the predictive solution and to defend the design decisions behind it.

2 Your objective

Deliver an **end-to-end, reproducible ML solution** that, given a pump's sensor history up to the present, forecasts how close it is to failure — early enough and reliably enough to act on.

What we are really assessing. This is an architect role. We care far more about *how you frame the problem, interrogate the data, engineer features, validate honestly, and reason about trade-offs* than about squeezing out a leaderboard score. A clear, well-argued, reproducible solution beats an opaque high-accuracy one.

3 The dataset

Everything is in [data/](#) . Full per-column definitions are in [docs/data_dictionary.md](#) — read it before you model.

FILE	WHAT IT IS
data/train.csv	Run-to-failure sensor history for 80 training pumps , long format, one row per operating cycle (~1 hour). ~16.6k rows, 20 columns.
data/train_units.csv	One row per training pump: commissioning date, number of cycles recorded, and the event flag (see §4).
data/sample_test_input.csv	A small illustrative example showing the exact schema your inference code will receive at the live test (§7). Two pumps, relabelled, no failure point.

This is production historian data — treat it as such. It was exported from a real SCADA system serving a mixed fleet. Channel quality, relevance and completeness are *not* guaranteed. Part of the job is deciding what to trust. We are not going to tell you which signals matter.

4 How failure is encoded

There is no explicit "failure" column. Failure is encoded through the `event` flag in `train_units.csv` :

EVENT	MEANING
1 · failed	The pump ran to failure . Its last recorded cycle is the failure point . 72 of the 80 units.
0 · censored	Monitoring stopped while the pump was still healthy . The true failure cycle is <i>unknown</i> and lies somewhere after the last recorded cycle. 8 of the 80 units.

The censored units are deliberate, and they are a real-world fact of reliability data. How you treat them is part of what we are evaluating — assuming their last cycle is a failure will silently poison your labels.

5 Choose your framing

We are giving you the raw run-to-failure data, not a pre-labelled target. **How you frame the prediction problem is your decision to make and to defend.** Reasonable framings include:

Option A

Remaining Useful Life

Regress the number of cycles left before failure. Direct, continuous, intuitive to operators.

Option B

Failure-within-window

Classify whether failure occurs within the next N cycles. Maps cleanly to an alerting threshold; needs care on imbalance and calibration.

Option C**Survival / hazard**

Model time-to-event directly — naturally handles the censored units.

Option D**Your own**

Something else entirely, if you can justify it. We want your reasoning, not a prescribed answer.

Pick one and own it. Be ready to explain why it fits this data, this business problem, and this fleet size — and what you traded away.

6 What to build

Submit a single Git repository (this one, extended) containing:

DELIVERABLE	NOTES
Reproducible pipeline	Data prep → feature engineering → training → evaluation. A fresh clone must run it from a documented command. Pin dependencies; seed randomness.
Trained model artifact	Saved weights / serialized model, loadable by your inference code without retraining.
Inference interface	A script or function that takes a CSV in the documented schema and emits predictions — see §7 for the exact contract.
Evaluation	Honest validation with unit-level splits (no leakage across a pump's own timeline), the metrics you chose, and why.

DELIVERABLE	NOTES
README	How to set up, train, evaluate, and run inference. Plus a short design write-up : framing, architecture, key decisions, assumptions, and what you would do with more time.

Any language is fine; Python is expected. Any libraries, classical or deep learning. A simple model you understand and can defend is entirely acceptable.

7 The live inference test

In the final 15 minutes of your presentation we will hand you a CSV of **pumps you have never seen** — same schema as `sample_test_input.csv`, each truncated at some point before failure — and run your inference code live.

Output contract

Whatever framing you chose in §5, your inference must produce, per unit, a **predicted Remaining Useful Life** — estimated cycles from the last observed cycle to failure. This is the common yardstick we score on. You may add columns (failure probability, confidence interval, failure-mode guess) — richer output is welcome.

```
# expected: a predictions CSV, one row per unit
unit_id,predicted_rul
PUMP_T01,57
PUMP_T02,33
...
```

Build this against `sample_test_input.csv` so it is a clean, one-command run on the day. It must run on CPU in a few minutes — no GPU, no internet.

If it does not run, we cannot score it. A working modest model beats a brilliant one that breaks on unfamiliar input. Rehearse the live command end-to-end before the call.

8 The presentation

One hour, with the team. Slides optional — a walkthrough of the repo and your reasoning is just as good.

TIME	SEGMENT
45 min	Architecture, data understanding, problem framing, feature engineering, the training process, model design, and your evaluation results. Expect us to probe decisions and trade-offs throughout.
15 min	Live inference on our held-out test pumps, followed by Q&A.

9 How we evaluate

We weight **reasoning and engineering judgment** above raw accuracy.

DIMENSION	WHAT A STRONG SUBMISSION SHOWS
Problem framing	A justified choice of target and horizon, tied to the business problem and the data you actually have.
Data understanding	Real EDA. You assessed channel quality and relevance, and noticed the structure in the operating data, rather than feeding all 20 columns in raw.
Feature engineering	Time-series features that fit degradation behaviour; sensible treatment of missing and noisy signals.
Modelling & architecture	A model appropriate to a 80-unit fleet. Complexity is justified, not assumed.
Validation honesty	Unit-level splits, no leakage across a pump's timeline, metrics that match the framing, and a clear read on where the model fails.
Reproducibility	A fresh clone runs. Pinned deps, seeds, a clear README.
Communication	You can defend every decision and name the limitations yourself.

10 Ground rules

- **AI assistants are allowed.** Use whatever tools you normally would. But you must understand and be able to defend every line — we will probe deeply.
- **Pretrained models and any libraries are fine**, as long as the pipeline runs offline on CPU for the live test.

- **Do not over-build.** Three days. A focused, well-reasoned solution is the goal — not a platform.
- **Document assumptions** rather than emailing questions. Stating a reasonable assumption clearly is itself a signal we value. If something is genuinely blocking, do reach out.
- **Commit incrementally** — we like to see how the work progressed.

11 Timeline & submission

- **Day 0** — you receive this repository.
- **~3 days** — design and build.
- **Presentation day** — push your final commit before the call, and present (1 hour, \$8). Put your write-up in the [README](#) and any extra deliverables under [submission/](#).

Confirm the presentation time with your CODVO.AI contact. Questions are welcome before then — just reply to the assignment email.

12 FAQ

Can I drop columns or units?

Yes — if you justify it. Removing a channel you have shown to be uninformative is good engineering. Removing data because it is inconvenient is not.

Deep learning or classical ML?

Your call. Both can score well here. We will ask why your choice fits a fleet of this size — have an answer.

How accurate does it need to be?

There is no pass/fail threshold. We assess the whole approach. An honest "here is where it works and here is where it does not" is worth more than an inflated number.

What if my inference output is richer than predicted RUL?

Great — include the extra columns. Just make sure `predicted_rul` is present so we can score the live test consistently.

CODVO.AI · Data Science Architect — Take-Home Assignment · Questions? Reply to your assignment email.